

Specifying a class

A class specification has two parts-

1. Class declaration
2. Class function definitions.

The general form of class declaration is

```
class class-name.  
{  
    private:  
        variable declarations;  
        function declarations;  
    public:  
        variable declarations;  
        function declarations;  
};
```

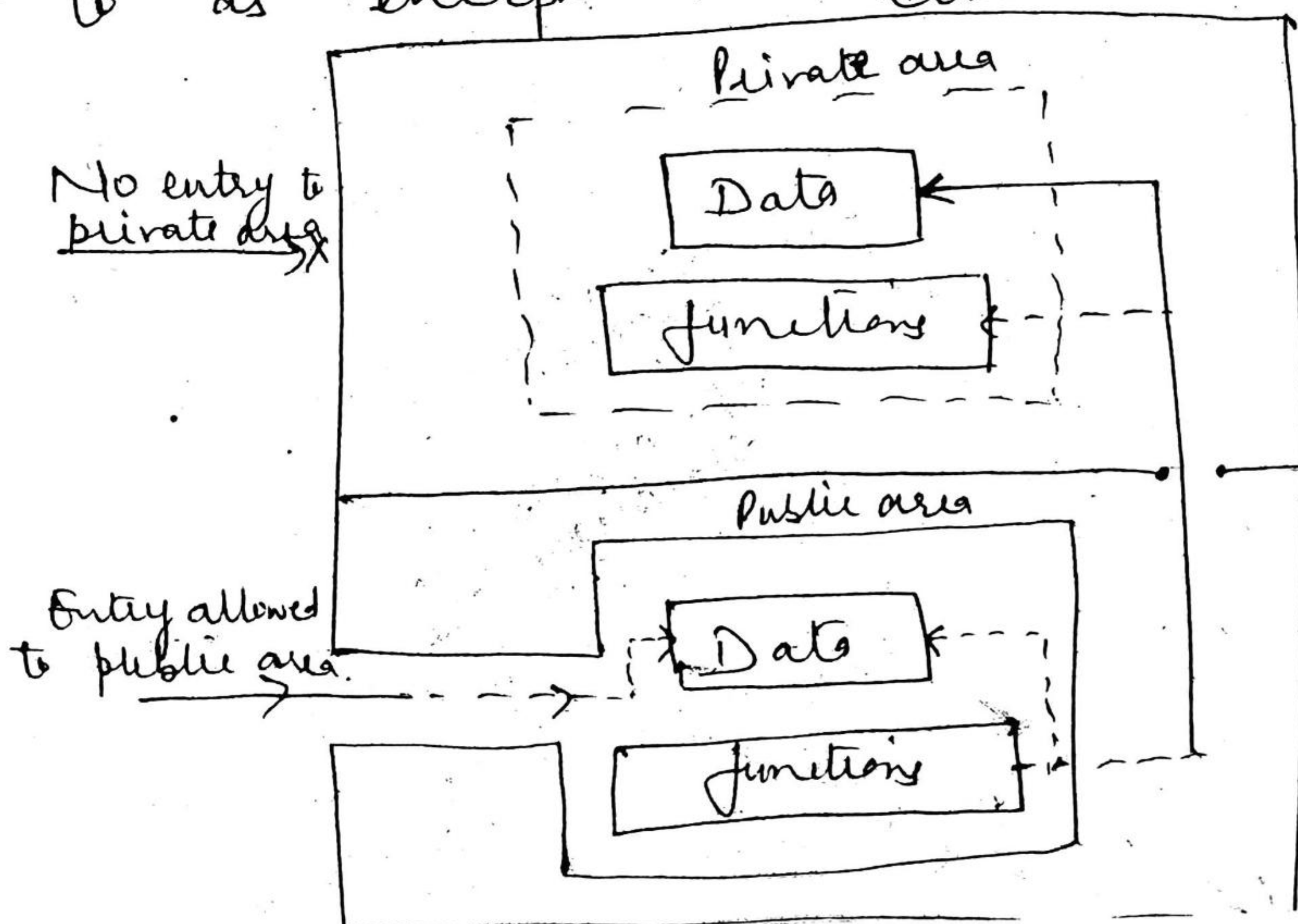

by default variable and functions are private.

If you want to make it public then you have to write the keyword public before declaration.

→ variables inside the class known as data members.

→ functions are known as member functions.

The binding of data and functions together into a single class-type variable is referred to as encapsulation. class



A Simple Class Example

```
class item
{
    int number;
    float cost;

public:
    void getdata (int a, float b);
    void putdata (void);
};
```

Creating Objects

item x; → memory for x is created.

eg item x, y, z;

* structure; provides only a template and does not create any memory space for the objects.

```
class item
{
    - - -
    - - -
    - - -
} x, y, z;
```

Accessing Class Members

The private data of a class can be accessed only through the member functions of that class.

* The main () cannot contain statements that access ~~member~~ data members directly.

Syntax of calling member function is.

`object-name.function-name (actual-arguments);`

e.g `x.getdata (100, 75.5);`

Defining Member functions

Member functions can be defined in two places:

- * Outside the class definition
- * Inside the class definition

Irrespective of the place of definition, the function should perform the same task.

Outside the class definition

* An important difference between a member function and a normal function is that a member function incorporates a membership identity label in the header. This 'label' tells the compiler which class the function belongs to.

return-type class-name :: function-name (arg. declaration)
{ function body }

:: → scope resolution operator

e.g.

```
void item :: getdata (int a, float b)
{
    number = a; cost = b;
}
```


Write the class definition

```
class item
```

```
{ int number;  
  float cost;
```

```
public:
```

```
    void getdata (int a, float b);
```

```
    void putdata (void) → inline function
```

```
{ cout << number << "\n";
```

```
  cout << cost << "\n";  
}
```




5.8 PRIVATE MEMBER FUNCTIONS

Although it is normal practice to place all the data items in a private section and all the functions in public, some situations may require certain functions to be hidden (like private data) from the outside calls. Tasks such as deleting an account in a customer file, or providing increment to an employee are events of serious consequences and therefore the functions handling such tasks should have restricted access. We can place these functions in the private section.

A private member function can only be called by another function that is a member of its class. Even an object cannot invoke a private function using the dot operator. Consider a class as defined below:

```
class sample
{
    int m;
    void read(void); // private member function
public:
    void update(void);
    void write(void);
};
```

If **s1** is an object of **sample**, then

```
s1.read();           // won't work; objects cannot access
                     // private members
```

is illegal. However, the function **read()** can be called by the function **update()** to update the value of **m**.

```
void sample :: update(void)
{
    read(); // simple call; no object used
}
```


Arrays within a Class

```
const int size = 10;  
  
class array  
{  
    int a[size];  
public:  
    void setval(void);  
    void display(void);  
};
```

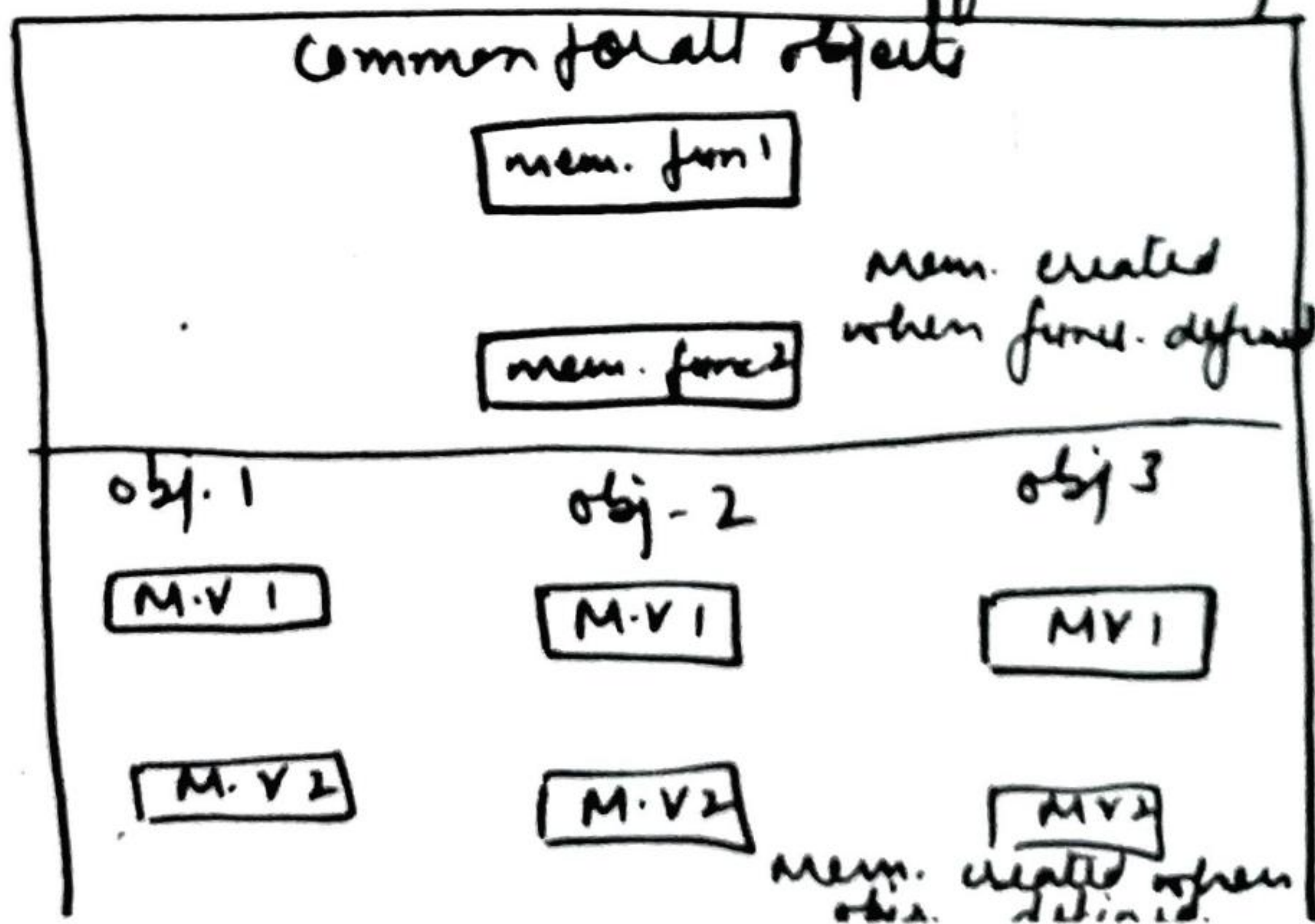
Program

```
#include <iostream.h>  
const m = 50;  
  
class ITEMS  
{  
    int itemcode[m];  
    float itemprice[m];  
    int count;  
public:  
    void init(void)  
    {  
        count = 0;  
    }  
    void getitem(void);  
    void displaySum(void);  
    void remove(void);  
    void displayItems(void);  
};
```


Memory Allocation for Objects:

The member functions are created and placed in the memory space only once when they are defined as a part of a class specification. Since all the objects belonging to that class use the same member functions, no separate space is allocated for member functions when objects are created.

- Only space for member variables is allocated separately for each object.
- because the member variables will hold different data values for different objects



Static Data Members

A static member variable has certain special characteristics. These are:

- It is initialized to zero when the first object of its class is created. No other initialization is permitted.
- Only one copy of that member is created for the entire class and it is shared by all the objects of that class, no matter how many objects are created.
- It is visible only within the class, but its lifetime is the entire program.
- * Static variables are normally used to maintain values common to the entire class.


```
#include <iostream.h>
```

```
class item
```

```
{ static int count;  
  int number;
```

```
public:
```

```
void getdata (int a)
```

```
{ number = a;  
  count ++;
```

```
}  
void getcount (void)
```

```
{ cout << "count : ";  
  cout << count << "\n";  
}
```

```
int item :: count; // definition of  
static data member
```

```
void main ()
```

```
{ item a, b, c;
```

```
  a.getcount();
```

```
  b.getcount();
```

```
  c.getcount();
```

```
  a.getdata (100);
```

```
  b.getdata (200);
```

```
  c.getdata (300);
```



```

count << "After reading data";
a. getcount ();           // display count
b. getcount ();
c. getcount ();
*
return 0;
}

```

O/P

count: 0

Count : 0

count : 0

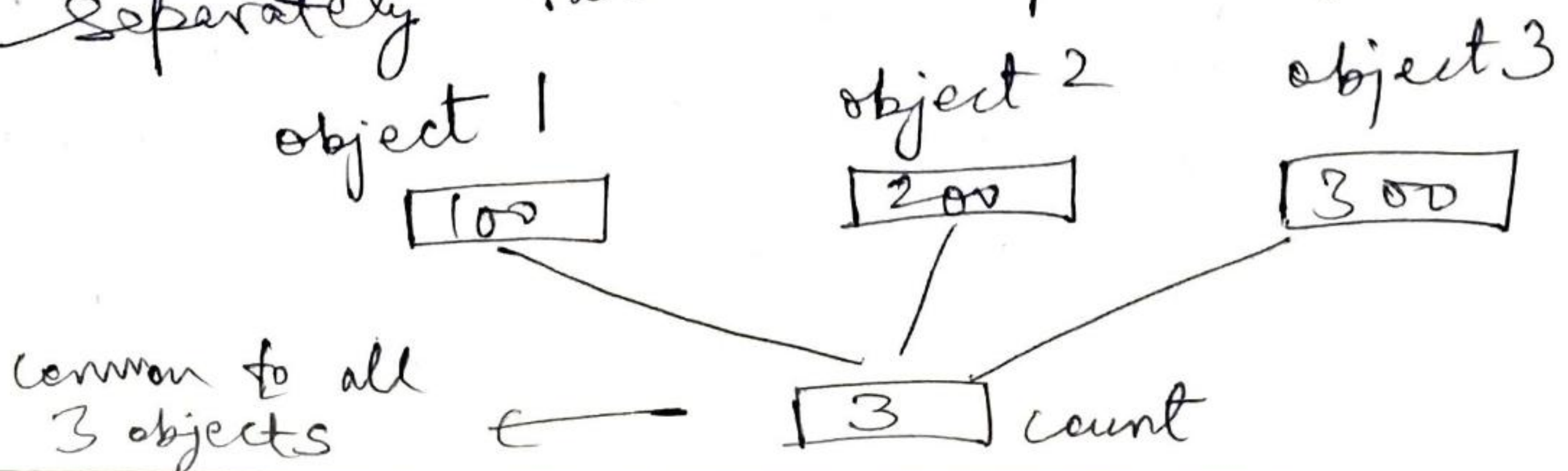
After reading data

count : 3

count : 3

count : 3

★ static member variable must be defined outside class definition bcz static data members are stored separately rather than part of object.



Static Member Functions

A member function that is declared static has the following properties:

- A static function can have access to only other static members (functions or variables) declared in the same class.

* A static member function can be called using the class name (instead of its objects)

E.g.

class-name :: function-name;

Program

```
#include <iostream.h>
class test
{
    int code;
    static int count;
public:
    void setcode(void)
    {
        code = ++count;
    }
    void showcode(void)
    {
        cout << "object no. : " << code << "\n";
    }
    static void showcoun(void)
    {
        cout << "count : " << count << "\n";
    }
};

int test :: count;

int main()
{
    test t1, t2;
    t1.setcode();
    t2.setcode();
    test :: showcoun();
    test t3;
    t3.setcode();
    test :: showcoun();
    t1.showcode(); t2.showcode(); t3.showcode();
    return 0;
}
```


Output:

count : 2

count : 3

object number : 1

object number : 2

object number : 3

code
1

t1

code
2

t2

code
3

t3

count

1, 2, 3

t1, t2, t3

* $\text{code} = ++\text{count}$;

is executed whenever `setcode()` function is invoked and the current value of `count` is assigned to `code`. Since each object has its own copy of `code`, the value contained in `code` represents a unique number of its object.

Indu Singh, AP, CSE, DTU